Software Architecture and Design Specification (SAD)

Project: Electronic Store Inventory Management System | **Version:** 1.0 | **Authors:** Karnati Uday Keerthan Reddy, K B Balavideshna, Kolluri Sruthi | **Status:** Approved

Revision History

Version	Author	Change Summary
1.0	Karnati Uday Keerthan Reddy, K B Balavideshna, Kolluri Sruthi	Complete Software Architecture & Design Specification incorporating Three-Tier Layered Architecture, UML Component & Sequence Diagrams, STRIDE Threat Model, and API Design.

Approvals

Role	Name	Approval Status
Course Instructor / Evaluator	Shravya S Madhusudan	Approved
Architecture & QA Lead	Karnati Uday Keerthan Reddy, K B Balavideshna, Kolluri Sruthi	Signed Off

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed system design of the Electronic Store Inventory Management System. It defines the three-tier layered decomposition, component interfaces, architectural rationale, STRIDE security threat model, dynamic UML sequence interactions, and HTTP/service API contracts.

1.2 Scope

Covers the end-to-end architecture of the retail electronics store web application: user authentication and role-based access enforcement (Admin, Manager, Staff), product catalog and supplier management, multi-item POS sales checkout and invoice generation, customer returns/refunds, low-stock threshold alerting, purchase order replenishment (full and partial delivery), date-range analytical reporting with CSV/PDF export, audit logging, and SQLite backup automation.

1.3 Audience

Software developers, QA and test engineers, security auditors, course instructors, and maintenance teams.

1.4 Definitions

Term / Acronym	Definition
Layered Architecture	Three-tier structural pattern separating Presentation, Business Logic, and Data Access responsibilities.
ADR	Architecture Decision Record — formal justification of a key architectural choice and rejected alternatives.
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege threat model.
ORM	Object-Relational Mapping (SQLAlchemy) — maps Python domain classes to relational database tables.
RBAC	Role-Based Access Control — server-side permission guards restricting endpoints to Admin, Manager, or Staff.

2. Document Overview

2.1 How to Use This Document

Section 3 presents the high-level system architecture, UML Component Diagram, component responsibilities, architectural pattern justification, technology stack, risks, requirement traceability, and STRIDE security architecture. Section 4 details the internal package design, UML Sequence Diagrams, component API specifications, error handling/audit logging strategy, and UX design.

2.2 Related Documents

• Software Requirements Specification (docs/SRS_Document.pdf, Documents/srs_table.pdf, Documents/security_requirements_specifications.pdf)
• Software Test Plan (docs/Test_Plan_Document.pdf, Documents/test_plan.pdf)
• Testing & Coverage Deliverables (docs/unit_test_report.pdf, docs/system_validation_report.pdf, docs/code_coverage_report.pdf, docs/branch_coverage_report.pdf)

3. Architecture

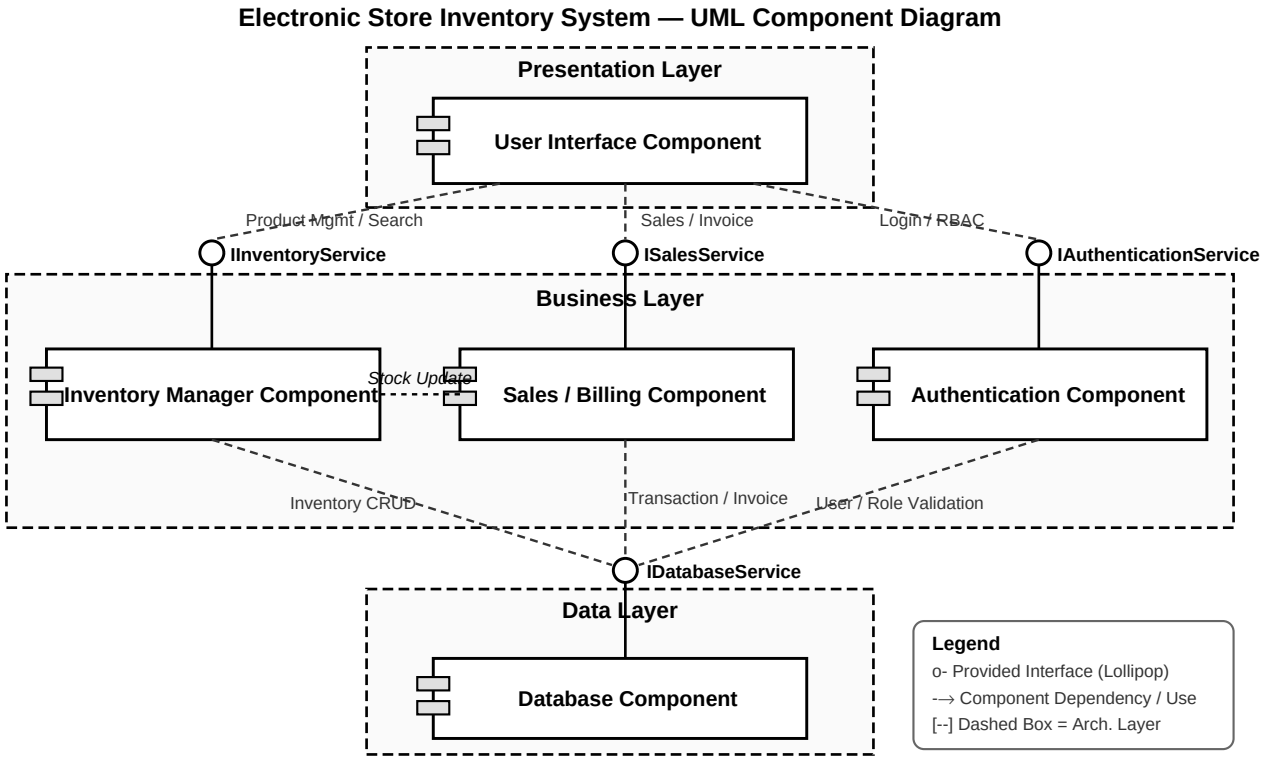
3.1 Goals & Constraints

Architectural Goals: (1) *Transactional Integrity* — guarantee ACID atomicity across multi-item sales, returns, and purchase order deliveries so stock and financial records never desynchronize (NFR-005); (2) *Low Latency* — complete checkout transactions within 2 seconds (NFR-001) and product searches within 1 second (FR-008); (3) *Modular Maintainability* — isolate Presentation, Business, and Data layers so inventory, billing, and reporting modules evolve independently (NFR-008); and (4) *Defense-in-Depth Security* — enforce salted bcrypt password storage and server-side RBAC on every route (NFR-003, SEC-001–SEC-016).
Constraints: Single-store deployment supporting up to 50 concurrent users and 10,000 SKUs (NFR-006); zero client-side installation across modern web browsers (NFR-007); cross-platform execution on macOS and Windows (both were used).

3.2 Stakeholders & Concerns

Stakeholder	Primary Architectural Concerns
Retail Store Staff	Fast, intuitive POS checkout, accurate real-time stock counts, clear validation messages, and printable customer invoices.
Store Managers	Immediate low-stock visibility, reliable purchase order creation and partial/full delivery tracking, and accurate date-range reports.
System Administrators	Centralized user/role lifecycle control, supplier catalog management, audit log traceability, and automated daily backups.
Security Auditors	Non-plaintext bcrypt credential storage, server-side RBAC enforcement, parameterized queries, and atomic financial records.
Developers & QA Team	Clean layer separation, deterministic dependency injection via Flask application factory (create_app), and high testability.

3.3 Component (UML) Diagram



3.4 Component Descriptions

Layer / Component	Provided Interface	Code Modules	Responsibilities
Presentation Layer: User Interface Component	HTTP / HTML & JSON Endpoints	app/presentation/routes/ (main.py, auth.py, supplier.py, purchase_order.py), authz.py, app/templates/*	Renders Jinja2 HTML views, parses form/query/JSON payloads, enforces @login_required, @admin_required, and @manager_required guards, and delegates business rules to the Business Layer.
Business Layer: Inventory Manager Component	IInventoryService	app/presentation/routes/main.py (catalog & report handlers), app/business/purchase_order.py, app/business/report_exporter.py	Manages product CRUD, unique SKU validation, category/stock-status search and filtering, low-stock threshold alerts, supplier linking, purchase order lifecycle, and CSV/PDF report generation.
Business Layer: Sales / Billing Component	ISalesService	app/business/sales.py (create_sale, process_return, restock_product, restock_inventory)	Validates item quantities and available shelf stock, calculates exact Decimal subtotals, 5% tax, and grand totals, executes atomic stock deductions/rollbacks, generates invoices, and processes customer returns.
Business Layer: Authentication Component	IAuthenticationService	app/business/security.py, app/business/password_reset_email.py, app/presentation/routes/auth.py	Hashes and verifies passwords using bcrypt (72-byte UTF-8 boundary check), manages Flask-Login sessions, issues signed itsdangerous password-reset tokens, and governs user role permissions.
Data Layer: Database Component	IDatabaseService	app/data/models.py, app/data/extensions.py, scripts/backup_database.py	Encapsulates SQLAlchemy ORM models (User, Product, Supplier, Sale, SaleItem, ProductRestock, ProductReturn, PurchaseOrder, PurchaseOrderItem, AuditLog), ACID transaction commits/rollbacks, and verified SQLite backups.

3.5 Chosen Architecture Pattern and Rationale

Architecture Selection: We chose **Layered Architecture** for the Electronic Store Inventory Management System.
Architectural Choice: The system is structured into three layers — a Presentation Layer (User Interface Component), a Business Layer (Inventory Manager, Sales/Billing, and Authentication Components), and a Data Layer (Database Component). Components primarily communicate across defined layer boundaries through clearly defined interfaces, while business components may interact with one another where required by the system workflow.
Reason 1 — Matches System Complexity: Our system has well-defined, sequential business processes (login →

search/select product → process sale → update stock → generate invoice) that map naturally onto horizontal layers. Unlike a high-traffic system that needs independent scaling of services (better suited to Microservices), our inventory system is used by a single store with a small number of concurrent users (NFR-006: up to 50 concurrent users), making the operational simplicity of a layered design more appropriate than the added complexity of Microservices.

Reason 2 — Supports Modular Maintainability (NFR-008): Since our requirements explicitly call for modular architecture so that 'inventory, billing, and reporting modules can be updated independently' (NFR-008), the layered approach allows each Business Layer component (Inventory Manager, Sales/Billing, Authentication) to be modified or extended without altering the Presentation or Data layers, satisfying this requirement directly.

Security Advantage: By isolating the Authentication Component as a distinct Business Layer service that all Presentation Layer interactions rely on for session validation, the architecture enforces centralized access control at the point of session establishment (login), while individual business components (Inventory Manager, Sales/Billing) independently enforce role checks on each request via NFR-003. This directly supports NFR-003 (RBAC enforcement on all endpoints) and NFR-005 (atomic transactions) — the UI layer never accesses the database directly, reducing the attack surface for unauthorized data access.

Performance Benefit: Because all business logic (stock validation, invoice calculation, role checks) is centralized in the Business Layer and only one Data Layer component handles all database I/O, we can optimize database queries and caching in a single place. This directly supports NFR-001 (process a sale transaction within 2 seconds) by avoiding redundant or scattered database calls that a distributed (Microservices) architecture would introduce through network latency.

3.6 Technology Stack & Data Stores

Architectural Tier / Concern	Technology / Library	Purpose in System
Operating System	macOS and Windows (both were used)	Cross-platform development, automated testing, and local store workstation execution.
Programming Language	Python 3.14.6	Core implementation language across presentation, business services, ORM models, and scripts.
Presentation & Web Framework	Flask 3.1.3, Jinja2 3.1.6, Werkzeug 3.1.9	HTTP routing via modular Blueprints, server-side HTML templating, and request/response handling.
Authentication & Security	Flask-Login 0.6.3, bcrypt 5.0.0, itsdangerous 2.2.0	Session management, salted bcrypt password hashing, and time-limited signed password-reset tokens.
Data Access & Persistence	Flask-SQLAlchemy 3.1.1, SQLAlchemy 2.1.6, SQLite 3	ORM entity modeling, parameterized SQL execution, ACID transaction management, and relational storage.
Reporting & Document Export	Python csv module, ReportLab 5.1.0	Generation of downloadable CSV spreadsheets and formatted PDF reports for sales and stock movements.
Verification & Coverage	Pytest 9.1.1, pytest-cov 7.1.0, coverage.py 7.17.2	373 automated unit/integration/route tests achieving 97.70% line and 91.50% branch coverage.

3.7 Risks & Mitigations

Architectural Risk	Impact	Mitigation Strategy in Design
Mid-transaction failure during multi-item sale or PO receipt	High (Stock & invoice desynchronization)	All multi-item mutations execute inside a single SQLAlchemy transaction with explicit db.session.rollback() on any validation or database exception (NFR-005, SEC-013).
Unauthorized access to Admin/Manager endpoints by Staff	High (Privilege escalation)	Server-side @admin_required and @manager_required decorators on every sensitive route return HTTP 403 Forbidden regardless of UI visibility (FR-005, SEC-006).
Database file loss or corruption on store terminal	High (Data loss)	scripts/backup_database.py uses SQLite's native backup API and verifies every backup copy using PRAGMA integrity_check == ('ok',) (NFR-009, SEC-014).
Password database compromise	High (Credential exposure)	Passwords are validated for 72-byte UTF-8 boundaries and hashed with unique salts via bcrypt; plaintext passwords are never stored or logged (NFR-003, SEC-003).
Floating-point rounding drift in currency totals	Medium (Financial inaccuracy)	All prices, tax amounts (5%), subtotals, and refunds use Python Decimal(10, 2) with ROUND_HALF_UP quantization.

3.8 Traceability to Requirements

Requirement IDs	Architectural Layer & Interface	Implementing Component / Module
FR-001, FR-003, FR-008, FR-010, NFR-010	Presentation + Business (IInventoryService) + Data (IDatabaseService)	Inventory Manager Component (app/presentation/routes/main.py) & Product / AuditLog models (app/data/models.py)
FR-002, FR-004, FR-011, NFR-001, NFR-005, SEC-013	Business (ISalesService) + Data (IDatabaseService)	Sales / Billing Component (app/business/sales.py) & Sale, SaleItem, ProductReturn models
FR-005, FR-012, FR-015, NFR-003, SEC-001..006, SEC-011	Presentation + Business (IAuthenticationService)	Authentication Component (app/presentation/routes/auth.py, app/presentation/authz.py, app/business/security.py)
FR-006, FR-007, FR-013	Presentation + Business (IInventoryService) + Data	Supplier & Purchase Order Components (app/presentation/routes/supplier.py, purchase_order.py, app/business/purchase_order.py)
FR-009, FR-014	Presentation + Business (IInventoryService)	Reporting & Export Services (app/presentation/routes/main.py, app/business/report_exporter.py)
NFR-009, SEC-014	Data Layer / Operational Utility	Database Backup Utility (scripts/backup_database.py)

3.9 Security Architecture (STRIDE Threat Modeling)

STRIDE Category	Threat Scenario	Architectural Mitigation & Control Reference
Spoofing	Attacker attempts to access POS or admin routes without logging in or by forging a password reset link.	Flask-Login session authentication required on protected routes (SEC-001); bcrypt password verification (SEC-003); cryptographically signed itsdangerous reset tokens with 30-min expiry (SEC-011).
Tampering	Unauthorized modification of product stock, prices, or finalized sales invoices.	Parameterized SQLAlchemy ORM queries prevent SQL injection (SEC-009); finalized invoices have no edit endpoint and corrections require audited ProductReturn / ProductRestock records (SEC-013).
Repudiation	User denies modifying product stock, changing prices, or deleting a catalog item.	AuditLog table records actor user_id (derived from server-side current_user, preventing spoofing), action type, target product, and UTC timestamp retained for ≥ 6 months (FR-010, NFR-010, SEC-010).
Information Disclosure	Exposure of plaintext passwords, payment card details, or internal stack traces/SQL text.	Salted bcrypt hashes only (SEC-003); no card numbers/CVV stored (SEC-015); generic user error messages with internal details logged only to current_app.logger (NFR-012, SEC-012); TLS 1.2+ in production (NFR-011, SEC-008).
Denial of Service	Oversized bcrypt password inputs or mid-transaction crashes leaving database locks or corrupted state.	72-byte UTF-8 password length guard in security.py; atomic transaction rollback on exceptions (NFR-005); daily verified SQLite backups (NFR-009).

STRIDE Category	Threat Scenario	Architectural Mitigation & Control Reference
Elevation of Privilege	Staff user invokes Manager (/purchase-orders, /reports/*) or Admin (/users, /suppliers, product delete) endpoints directly.	Server-side @admin_required and @manager_required decorators validate current_user.role on every request and return HTTP 403 Forbidden (FR-005, NFR-003, SEC-006).

4. Design

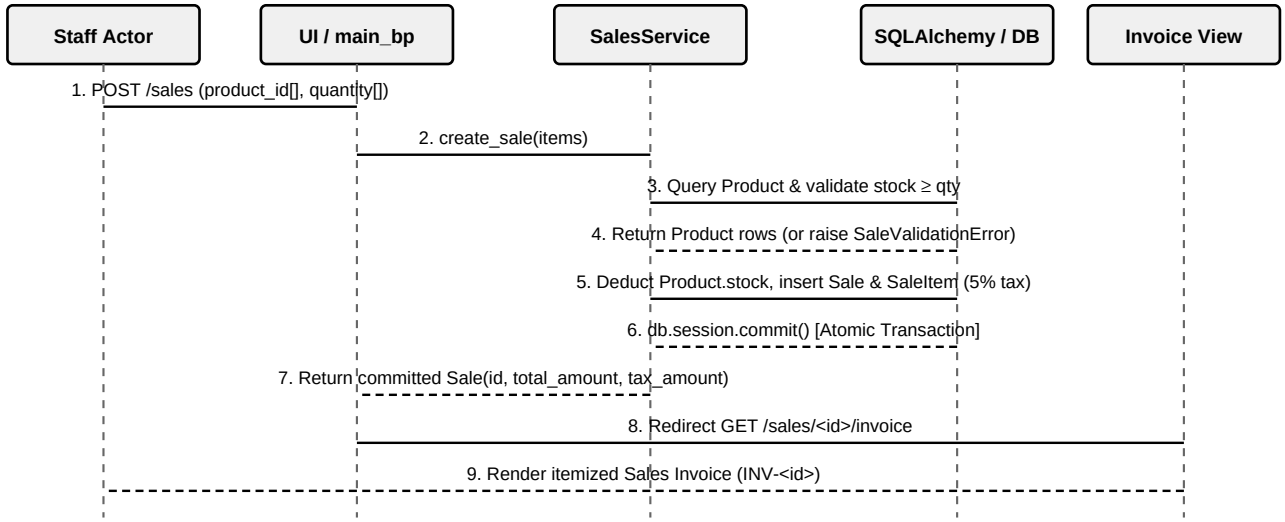
4.1 Design Overview

The application is organized under the top-level app/ package using the Flask Application Factory pattern (create_app() in app/__init__.py). Presentation blueprints (main_bp, auth_bp, supplier_bp, purchase_order_bp) are registered in app/presentation/routes/__init__.py. Domain logic is encapsulated in stateless service functions under app/business/, and persistent entities are modeled in app/data/models.py using SQLAlchemy declarative mappings.

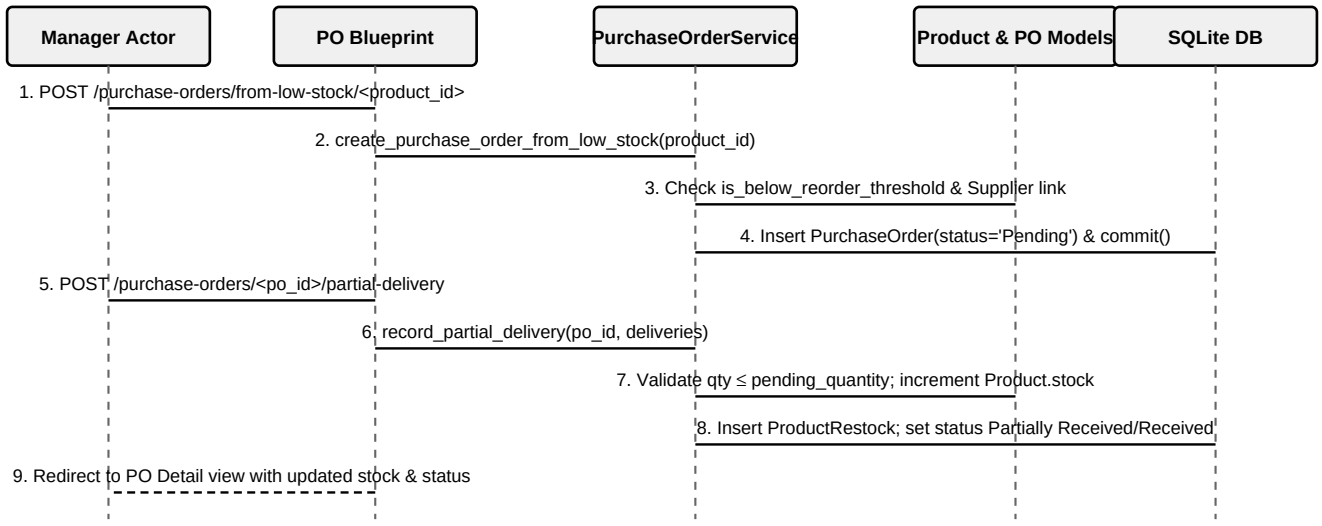
4.2 UML Sequence Diagrams

In accordance with the SAD template requirement (at least 2 sequence diagrams covering 2 project-specific flows), Figure 4.1 illustrates the multi-item POS sale checkout and invoice generation flow, and Figure 4.2 illustrates the low-stock purchase order creation and partial/full delivery receipt flow.

Sequence Diagram 1: Multi-Item POS Sale Checkout & Atomic Invoice Generation



Sequence Diagram 2: Low-Stock PO Creation & Partial/Full Delivery Receipt Flow



4.3 API Design (Component Interface Specifications)

Table 4.1 defines the HTTP endpoint and service interfaces across all four core architectural components (exceeding the template minimum of 2 components).

Component	Endpoint / Service	Method	Request Parameters / Payload	Success Response	Error Responses
Sales & Billing (ISalesService)	/sales create_sale(items)	POST	Form/JSON: product_id[], quantity[] (positive integers)	302 Redirect to /sales/<id>/invoice (or 201 JSON with sale_id, subtotal, tax_amount, total_amount)	400 Bad Request / SaleValidationError: empty items, non-integer/<=0 qty, insufficient stock; 404 Product not found.
Sales & Billing (ISalesService)	/returns process_return(...)	POST	JSON/Form: sale_item_id or (product_id, quantity, refund_amount), return_id	201 Created JSON: return_id, product_id, quantity, refund_amount, updated stock	400 Validation Error: duplicate return_id, return qty > sold qty, invalid amount; 404 SaleItem/Product not found.
Inventory Manager (IInventoryService)	/products/create /products/<id>/edit	POST	Form: name, sku, category, price, stock, reorder_threshold, supplier_id	302 Redirect to / with flash 'Product created/updated successfully.' and AuditLog entry	400/409 Error: blank name, negative price/stock/threshold, duplicate SKU.
Inventory Manager (IInventoryService)	/reports/sales /reports/stock /reports/revenue	GET	Query: start_date (YYYY-MM-DD), end_date (YYYY-MM-DD)	200 OK HTML report (or CSV/PDF stream on /export/<format>)	400 Error banner: missing date, invalid date format, or start_date > end_date; 403 if Staff role.
Purchase Order Component	/purchase-orders/create /purchase-orders/<id>/receive /partial-delivery	POST	Create: supplier_id, product_id[], quantity[]. Partial: item_id[], delivered_quantity[]	302 Redirect / 200 JSON: PO created (Pending) or stock incremented & status Partially Received / Received	400 PurchaseOrderError: invalid supplier/product, qty ≤ 0, delivered_qty > pending_quantity, already Received; 403 if Staff.
Authentication (IAuthenticationService)	/login, /logout /users/create /reset-password/<token>	POST	Login: username, password. Create User: username, email, password, role	302 Redirect to / or /users with active session or updated user record	400/401/403/409: invalid credentials, deactivated user, password < 8 chars, duplicate username/email, expired token.

4.4 Error Handling, Logging & Monitoring

• **Domain Exception Hierarchy:** Business services raise explicit, typed exceptions (SaleValidationError in app/business/sales.py and PurchaseOrderError in app/business/purchase_order.py) carrying human-readable validation messages.
• **Transactional Rollback Guarantees:** Every database mutation block wraps db.session.commit() in try/except handlers that immediately invoke db.session.rollback() upon IntegrityError or SQLAlchemyError, preventing partial state writes.
• **Sanitized User Feedback vs. Server Logging:** Route handlers catch unexpected internal exceptions, log full diagnostic tracebacks exclusively to current_app.logger.exception(...), and present generic, safe error messages to the user (NFR-012, SEC-012).
• **Immutable Audit Trail:** Critical catalog and stock modifications write structured AuditLog records (action, product_id, product_name, user_id, details, created_at) retained for at least 6 months (FR-010, NFR-010).

4.5 UX Design

The user interface is designed for clarity and rapid retail operation across desktop and tablet viewports: (1) a sticky top navigation bar provides one-click access to POS & Inventory, Catalog Entry, Purchase Orders, Suppliers, Reports, and Admin User Management; (2) a top KPI summary strip surfaces listed SKU count, total shelf units, active low-stock alerts, and categories; (3) low-stock items are highlighted both in a top warning banner and inline table badges; (4) data tables use keyboard-scrollable regions (tabindex='0') and tabular monospace numerals for prices and SKUs; and (5) invoices include a dedicated print stylesheet triggered via Print Invoice.

4.6 Open Issues & Next Steps

• **Production HTTPS/TLS Termination:** Configure Nginx/Caddy reverse proxy with TLS 1.2+ certificates and Secure cookie flags during live store deployment (NFR-011, SEC-008).
• **Automated Login Lockout Counter:** Extend User model with failed_login_attempts and locked_until columns to enforce the 5-attempt / 15-minute lockout rule in SEC-004.
• **Scheduled Backup Restore Drills:** Pair scripts/backup_database.py with an OS cron/Task Scheduler job and automated restore verification script (NFR-009, SEC-014).

5. Appendices

5.1 Glossary: ACID, ADR, bcrypt, Blueprint, CSV, ORM, POS, PO, RBAC, RPO, SKU, SQLite, STRIDE, TLS.
5.2 References: (1) IEEE Std 42010 — Systems and Software Engineering: Architecture Description; (2) OWASP Application Security Verification Standard (ASVS); (3) Flask 3.1 & SQLAlchemy 2.1 Official Documentation.
5.3 Tools Used: Python 3.14.6, Flask, SQLAlchemy, Pytest, coverage.py, ReportLab, Git.